# Go Backend Rebuild Recommendation

## Repo snapshot (current state)

- Monorepo with two Ionic/React PWAs: `src/order` (customer) and `src/manage` (admin).
- NestJS API in `src/api` with MongoDB Atlas, SuperTokens auth, WebSocket events, and integrations (payments, email, web push, printers).
- Mobile apps rely on REST endpoints under `order-app/*` plus WebSocket events for real-time updates.
- Go GraphQL demo service is available in `src/go-graphql-demo` with a `menus` query.
- Local seed data is loaded from `docs/menu/*` into Mongo via `dev/seed-mongo.js`.

## Core backend responsibilities to port

- Auth/session: SuperTokens middleware and guards (`src/api/src/auth/*`).
- Order flow: preview orders, checkout, order status (`src/api/src/order-app/*`).
- Menu and restaurant data: menus, restaurants, origins, locations, campaigns (`src/api/src/menu/*`, `src/api/src/restaurant/*`, `src/api/src/origins/*`).
- Real-time events: Socket.IO gateway for order updates (`src/api/src/events/*`).
- Operational features: stations, printers, location settings, reports, POS (`src/api/src/stations/*`, `src/api/src/printers/*`, `src/api/src/location-settings/*`, `src/api/src/report/*`, `src/api/src/pos/*`).
- Integrations: email templates, web push, payments, Firebase admin (`src/api/src/email/*`, `src/api/src/web-push/*`, `src/api/src/payments/*`).

## Feasibility: NestJS to Go

- Feasible with a staged migration. The NestJS API is modular and maps cleanly to Go packages/services.
- MongoDB access uses the driver directly (no heavy NestJS-specific ORM), so porting queries is straightforward.
- WebSocket events can be replicated with Go (Gorilla/WebSocket or Socket.IO-compatible layer if needed).
- Middleware and validation logic must be reimplemented (logging, auth guards, request IDs).
- Existing mobile clients expect REST shapes (e.g., `ApiResponse<T>`). Keeping response contracts stable is the highest priority.
- A GraphQL read-path prototype already exists (menus list) to validate incremental migration.

## Core functionality feasibility table

| Core area (x) | Go feasibility (y) | Required changes / notes |
|---|---|---|
| Auth/session (SuperTokens) | Medium | Rebuild middleware/guards, cookies, and session refresh logic in Go; validate client SDK expectations. |
| Order app read APIs (menus, restaurant, origin, campaign) | Low | Straightforward MongoDB queries; keep DTO shapes aligned to mobile Zod schemas. |
| Preview order + pricing/discount logic | Medium | Recreate pricing, modifiers, tax, and campaign discount rules; add unit tests for price calc parity. |
| Checkout + order persistence | Medium | Map order schema and lifecycle updates; ensure idempotency and request correlation IDs. |
| Real-time events (Socket.IO) | Medium-High | Choose compatible Socket.IO server or migrate clients to plain WebSocket; preserve event names/payloads. |
| Admin/manage APIs (stations, printers, location settings, reports, POS) | Medium | Port controllers/services; watch for complex aggregation/report queries. |
| Payments integration | Medium-High | Re-implement providers, webhook validation, and retries; verify PCI-related flow handling. |

| Core area (x) | Go feasibility (y) | Required changes / notes |
|---|---|---|
| Email + web push | Low-Medium | Port templates and send pipelines; keep payload shapes and failure logging. |
| Observability + logging | Medium | Replace NestJS middleware logging with Go middleware; reintroduce Azure App Insights tracing and correlation IDs. |
| File/storage services | Medium | Rebuild upload endpoints, ACLs, and storage providers; confirm any CDN assumptions. |

## Go for the upcoming web app

- Go can comfortably serve both mobile and web clients using the same API contract.
- GraphQL is supported and already demoed for read paths; REST remains a stable fallback during migration.
- Web client needs the same real-time events (order status, station routing). Go can handle this with WebSockets and pub/sub.
- CORS and session management can be centralized in Go for both web and mobile.

## Proof of concept completed (local)

- Go GraphQL `menus` query serves the Order App via `VITE_GRAPHQL_ENDPOINT`.
- Order App menus list can switch between REST and GraphQL without changing UI code.
- Seeded menu data is sourced directly from `docs/menu/*` for local MVP testing.
- Deterministic IDs allow stable local URLs for demo and testing.

## Local MVP path (recommended)

1. Use `dev/seed-mongo.js` to load `docs/menu/*` into Mongo.
2. Run `dev/start-apps.sh` to launch Order, Manage, API, and Go GraphQL demo.
3. Validate the menus list page via the seeded URLs in `docs/demo-go-graphql-mobile.md`.

## Recommendation

- Proceed with a Go rebuild, but avoid a big-bang cutover.
- Start by duplicating read-only endpoints used by the Order App (`order-app/*` GETs), then move writes (preview order, checkout), then the admin dashboard endpoints.
- Maintain API contracts and align DTOs with existing Zod schemas in the mobile apps.
- Keep the NestJS service running during migration, with a routing layer (or feature flags) to direct traffic to Go incrementally.

## Risks and mitigations

- Contract drift: Define and freeze API contracts (OpenAPI or shared schema) and add contract tests.
- Real-time parity: The Socket.IO event model must be preserved or a compatible adapter provided.
- Auth/session parity: Validate SuperTokens behavior and cookie/session handling across clients.

## Suggested next steps

1. Add GraphQL equivalents for restaurant/location/origin/campaign reads.
2. Add GraphQL `menu` details query and switch `useMenu.ts` under the flag.
3. Port preview order + checkout mutations to Go and validate pricing parity.
4. Introduce real-time events and admin APIs after read/write parity is proven.